

Git Assignment 1

Enterprise Source Code Management

Git-based source control for a 50-developer online banking platform

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course / Subject	Java Full Stack
Instructor	Mr. Pravin
Submission date	October, 01, 2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Business Scenario and Objectives.....	3
2.1 Scenario.....	3
2.2 Objectives	3
3. Stakeholders.....	3
4. Current Challenges and Risks	4
5. Assumptions and Constraints	4
5.1 Given Requirements (from the assignment)	4
5.2 Assumptions (made by the author)	4
5.3 Constraints.....	5
6. Proposed Solution (Step-by-Step Approach).....	5
6.1 Repository Structure	5
6.2 Branching Strategy	5
6.3 Commit Standards	6
6.4 Merge Process and Conflict Handling	7
6.5 Release Branches and Release Workflow	9
6.6 Rollback and Recovery Procedures	10
7. Implementation Plan.....	11
8. Measurable Success Criteria	12
9. Recommendations and Lessons Learned	12
9.1 Recommendations.....	12
9.2 Lessons Learned	12

1. Executive Summary

Problem. A software company has 50 developers building an online banking platform with five modules: authentication, payments, reporting, notifications and customer management. Developers overwrite each other's changes, releases are delayed, and there is no consistent branching strategy.

Solution. Adopt a single, shared Git workflow built on one repository, a simplified Git Flow branching model, a standard commit message format, pull requests with mandatory reviews, dedicated release and hotfix branches, and written recovery procedures for mistakes.

Expected result. No overwritten work, predictable releases, a traceable history for banking audits, and fast, safe recovery when something goes wrong. Section 8 defines how success will be measured.

2. Business Scenario and Objectives

2.1 Scenario

Five teams work in parallel on the same banking platform. Without agreed rules, they push to the same code, resolve conflicts differently, and release at different times. In a bank this is more serious than in an ordinary product: a bad change to payments or authentication affects real money and customer security.

2.2 Objectives

#	Objective	Why it matters
O1	Stop developers overwriting each other's work	Lost work wastes time and causes bugs
O2	Introduce one consistent branching strategy	Everyone knows where to work and where code goes
O3	Make releases predictable and on time	Delayed releases hurt the business and customers
O4	Keep production stable and auditable	Banks must show who changed what, when and why
O5	Make recovery from mistakes quick and safe	Accidental commits and bad releases will happen
O6	Keep the process simple enough that 50 developers follow it	A process nobody follows is worse than none

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, how the code is managed.

Stakeholder	Role	What they need from Git
Developers (50)	Write and change code	Clear branches, few conflicts, simple commands
Team leads (5)	Lead each module team	Visibility of work, control over what is merged
QA / Testers	Test before release	Stable release branches to test against
Release manager	Plans and approves releases	Clear release branches, tags and version history
DevOps engineers	Build, deploy and run pipelines	Predictable branches that trigger automated builds
Security team	Protect customer data and code	Reviews on sensitive code, no secrets in history
Compliance / Audit	Check regulatory rules	Traceable history: who, what, when, why
Product owner / Business	Decide features and dates	On-time, reliable releases
Customers (bank users)	Use the platform	A stable, secure banking application

4. Current Challenges and Risks

#	Challenge	Impact	Risk
C1	Developers overwrite each other's changes	Lost work, repeated effort, hidden bugs	High
C2	No consistent branching strategy	Confusion about where code goes; messy history	High
C3	Delayed releases	Missed deadlines; unstable, rushed fixes	High
C4	Unreviewed code reaches shared branches	Defects or vulnerabilities in banking code	High
C5	No commit standard	Unclear history; hard to trace or audit changes	Medium
C6	Merge conflicts handled ad hoc	Wrong code kept; broken builds	Medium
C7	No recovery procedure for mistakes	Long outages; panic fixes; possible data exposure	High
C8	Secrets (passwords, keys) committed by accident	Security breach and compliance failure	High

Risk level is a judgement based on the likely effect on a bank (security, money, reputation). It is not measured data.

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**, so the two are never confused.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	50 developers work on an online banking platform
R2	Five modules: authentication, payments, reporting, notifications, customer management
R3	Design the repository structure
R4	Create a branching model / strategy
R5	Define commit standards
R6	Handle merge conflicts (merge process)
R7	Maintain release branches and define a release workflow
R8	Document recovery procedures for accidental commits (rollback approach)
R9	Deliver a report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	GitHub is the hosting platform (GitLab or Bitbucket work the same way)	Menu names change; the process does not
A2	Each of the five modules is owned by one team of about 10 developers	Review rules would need adjusting
A3	The platform is released on a regular schedule (assumed every 2 weeks)	Release cadence in Section 6.5 changes
A4	A CI pipeline (automated build and tests) is available or will be set up	Automated checks would become manual reviews

#	Assumption	Effect if wrong
A5	Current problem numbers (conflicts, delays) are not known, so targets in Section 8 are illustrative	Targets should be re-based on real data
A6	Developers already know basic Git (commit, push, pull)	Extra training time needed

5.3 Constraints

- **Regulation:** banking rules require an audit trail, access control and separation of duties.
- **Security:** payments and authentication code needs stricter review than other modules.
- **People:** 50 developers must adopt the process, so it must stay simple.
- **Time:** the process must be introduced without stopping ongoing development.
- **Scope:** this is a design and planning exercise; no real company systems are changed.

6. Proposed Solution (Step-by-Step Approach)

The solution has six parts, taken in order. Each part solves one or more challenges from Section 4.

6.1 Repository Structure

Decision: use one shared repository (a "monorepo") with one folder per module.

Simple explanation: think of one building with five departments on separate floors. Everyone uses the same entrance and rules, but each floor has its own owner.

```
banking-platform/
  auth/           <- authentication module
  payments/       <- payments module
  reporting/      <- reporting module
  notifications/  <- notifications module
  customer-mgmt/  <- customer-management module
  shared/         <- common libraries used by all modules
  docs/          <- documentation and this workflow
  .github/
    CODEOWNERS    <- which team must review which folder
    pull_request_template.md
  .gitignore      <- keeps secrets and build files out
  README.md
```

Why one repository: a change that touches several modules (for example payments and notifications) can be made and reviewed together, and one branching strategy applies everywhere.

Trade-off: everyone can see all code. This is handled with a **CODEOWNERS** file, which forces the owning team (and security for auth and payments) to approve changes to their folder. The alternative, one repository per module, gives stricter access but makes cross-module changes harder.

6.2 Branching Strategy

Decision: use a simplified Git Flow.

Simple explanation: a branch is a separate copy of the code where you can work without disturbing others. Think of it as a draft copy of a document that is merged into the final version only after review.

Branch	Purpose	Created from	Merged into	Lifetime
main	Live production code. Every commit is a released version.	n/a	n/a	Permanent
develop	Integration branch where finished features come together	main	release/*	Permanent
feature/*	One new feature or task	develop	develop	Short (days)
bugfix/*	Non-urgent bug fix	develop	develop	Short (days)

Branch	Purpose	Created from	Merged into	Lifetime
release/*	Prepares and stabilises a release	develop	main and develop	About 1 week
hotfix/*	Urgent fix for a problem live in production	main	main and develop	Hours to a day

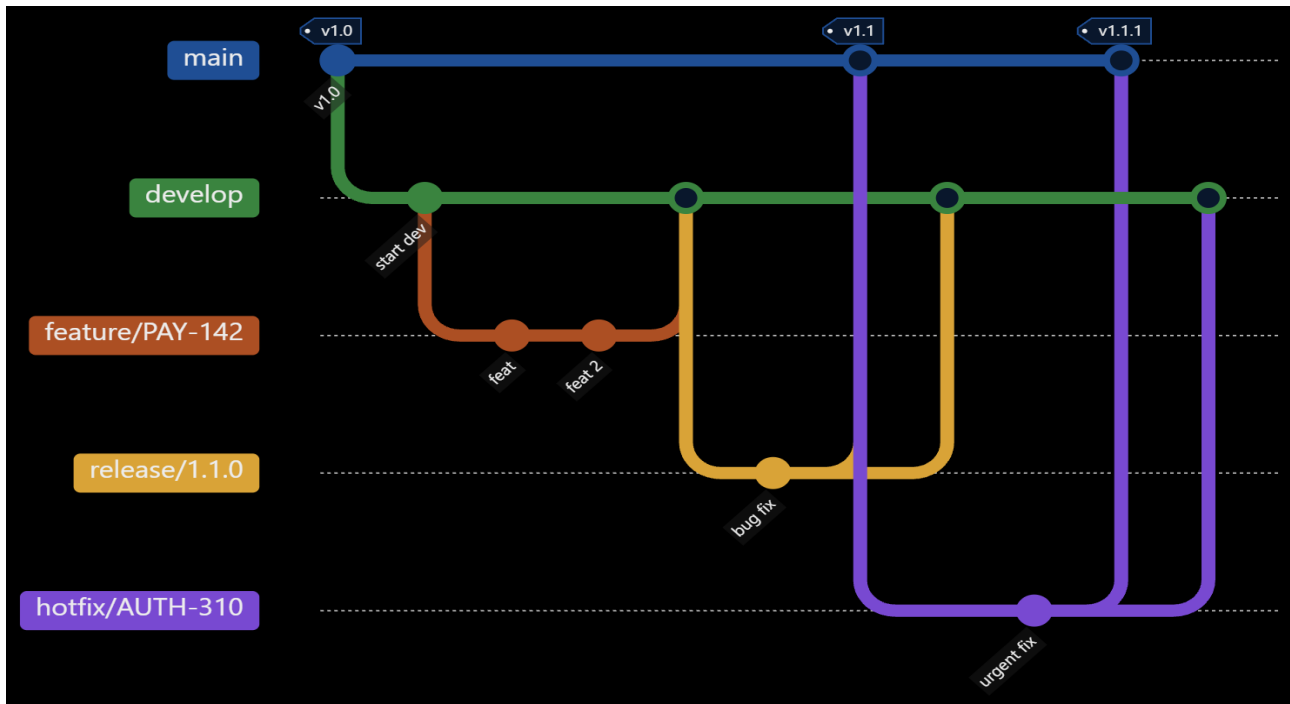


Diagram 1: Branching Model

Rules

- **main and develop are protected:** nobody pushes directly; changes arrive only through pull requests.
- **Naming:** type/TICKET-short-description, for example feature/PAY-142-upi-refund.
- **Keep feature branches short** (ideally under 3 days) to reduce conflicts. This is the main weakness of Git Flow, so it is controlled by rule.

Commands

```
git checkout develop
git pull origin develop
git checkout -b feature/PAY-142-upi-refund
# ...work and commit...
git push -u origin feature/PAY-142-upi-refund
```

Why not the alternatives?

Model	Why not chosen
GitHub Flow (only main + short branches)	No release branches, which the assignment requires; weaker for approval-heavy banking releases
Trunk-based development	Needs very strong automated tests and feature flags; hard to justify without release branches and formal approvals

6.3 Commit Standards

Simple explanation: a commit is a saved snapshot with a message. A good message lets anyone understand a change a year later, which matters for audits.

Format (Conventional Commits):

```
<type>(<scope>): <short description>
```

```
<optional body: what and why>
```

```
Refs: PAY-142
```

Type	Use for
feat	A new feature
fix	A bug fix
docs	Documentation only
refactor	Code cleanup with no behaviour change
test	Adding or changing tests
chore	Build, config or tooling

Example:

```
feat(payments): add UPI refund endpoint
```

```
Allows customers to request refunds on failed UPI payments.
```

```
Refs: PAY-142
```

Commit rules

- Subject line is 72 characters or fewer, in the present tense ("add", not "added").
- One logical change per commit; commit often.
- Every commit references a ticket ID.
- **Never commit** passwords, API keys or customer data; `.gitignore` and a secret-scanning check help prevent this.
- Commits to payments and auth are signed so the author can be verified.

6.4 Merge Process and Conflict Handling**Merge process (pull request flow)**

1. Developer finishes work on a feature branch and syncs it with `develop`.
2. Developer opens a pull request (PR) into `develop`.
3. Automated checks run (build, tests, secret scan).
4. At least 1 reviewer approves (2 for payments and auth, via CODEOWNERS).
5. The PR is merged using **squash merge**, giving one clean commit per feature.
6. The feature branch is deleted.

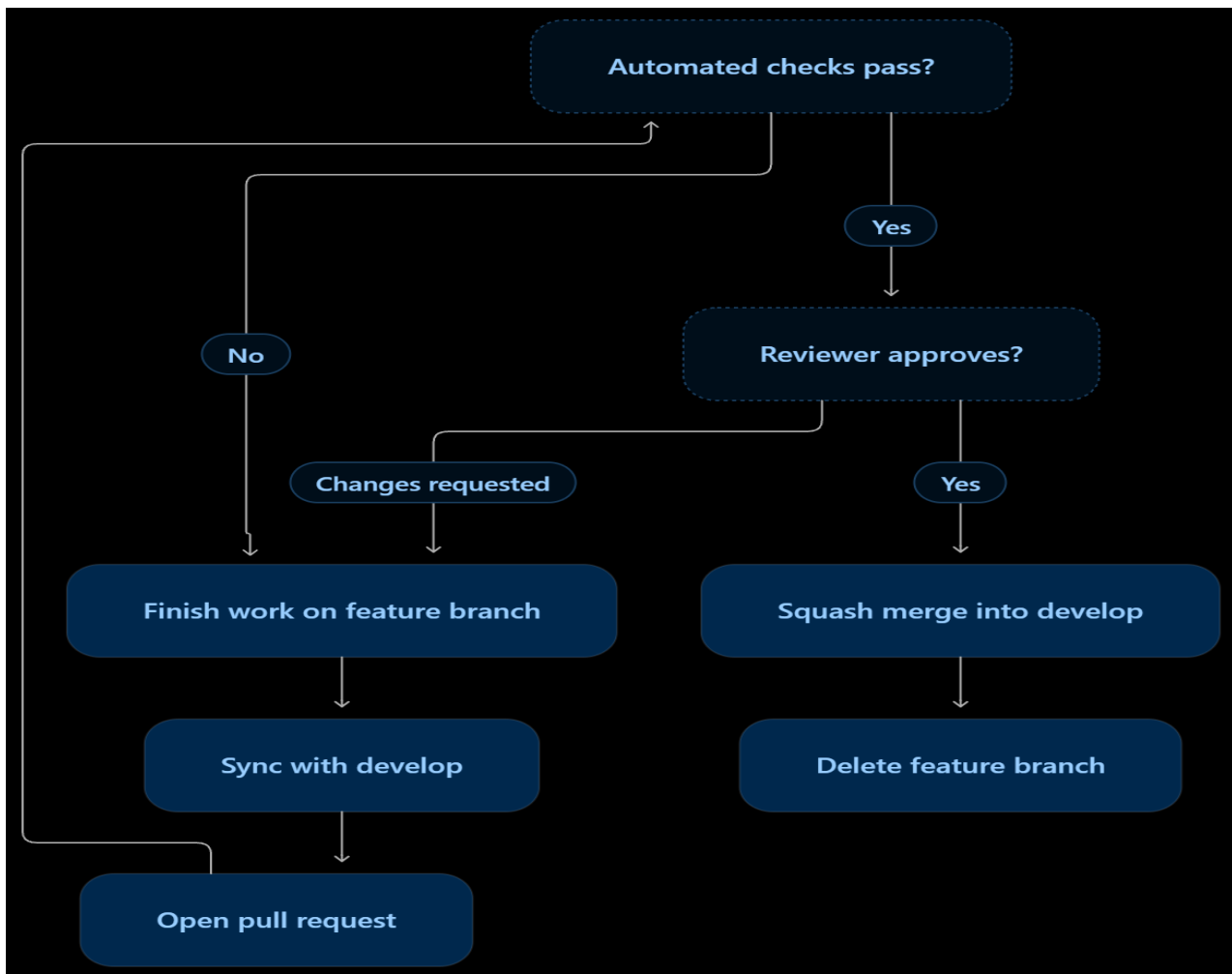


Diagram 2: Pull Request and merge flow

What is a merge conflict?

A conflict happens when two people change the same lines of the same file. Git cannot choose, so a human must decide which version to keep.

Preventing conflicts

- Keep branches short and PRs small.
- Sync with `develop` at least once a day.
- Module ownership (CODEOWNERS) means teams mostly work in different folders.

Resolving a conflict

```

git checkout feature/PAY-142-upi-refund
git fetch origin
git rebase origin/develop      # or: git merge origin/develop
# Git stops and marks conflicts in the file:
# <<<<<< HEAD
# (their version)
# =====
# (your version)
# >>>>>> your-commit
# 1. Edit the file: keep the correct code, delete the markers
git add <file>
git rebase --continue
# 2. Run the tests
git push --force-with-lease    # safe force push, own branch only

```

Rule: if the conflict is in code you do not understand, talk to the other developer before resolving. Never accept one side blindly. Force pushing is allowed only on your own feature branch, never on `main`, `develop` OR `release/*`.

6.5 Release Branches and Release Workflow

Simple explanation: a release branch is a "freeze" of the code. While QA tests it, only bug fixes are allowed, so new features cannot break the release, and the team keeps building the next version on `develop`.

Release workflow

1. Create `release/1.4.0` from `develop` when the planned features are done.
2. Freeze: only bug fixes go into the release branch.
3. QA tests the release branch (regression, security, performance).
4. Release manager and compliance approve.
5. Merge `release/1.4.0` into `main` and tag it `v1.4.0`.
6. Merge `release/1.4.0` back into `develop` so fixes are not lost.
7. Deploy the tagged version to production; delete the release branch.

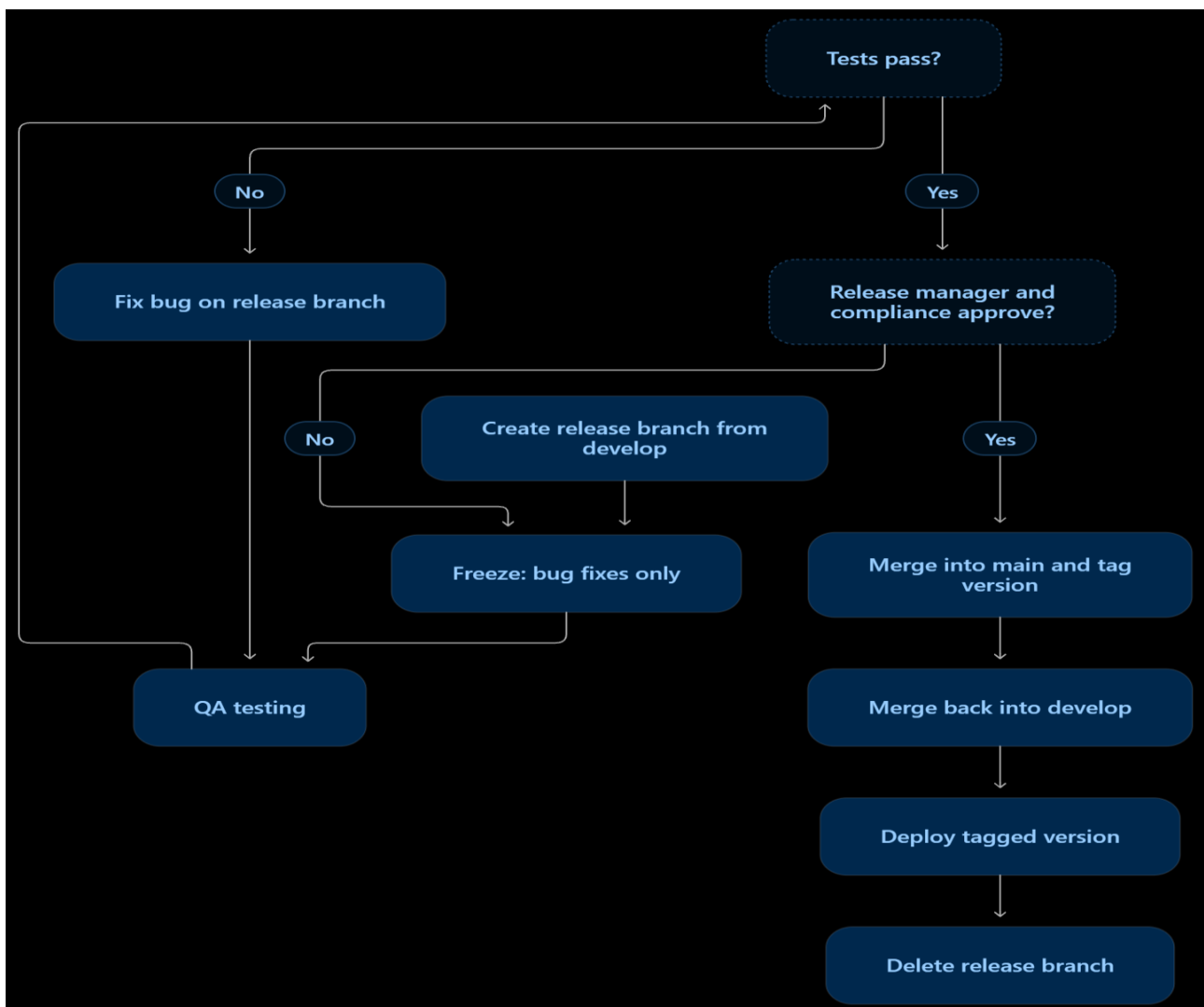


Diagram 3: Release workflow

```
git checkout develop && git pull
git checkout -b release/1.4.0
# ...bug fixes only, after QA...
git checkout main && git pull
git merge --no-ff release/1.4.0
git tag -a v1.4.0 -m "Release 1.4.0"
git push origin main --tags
git checkout develop
git merge --no-ff release/1.4.0
git push origin develop
```

Hotfix workflow (urgent production bug)

```
git checkout main && git pull
git checkout -b hotfix/AUTH-310-login-lockout
# ...fix, test, PR with expedited review...
# merge into main, tag v1.4.1, then merge into develop too
```

Versioning: use MAJOR.MINOR.PATCH (for example 1.4.1). Features raise MINOR, hotfixes raise PATCH.

6.6 Rollback and Recovery Procedures

Mistakes will happen. The right fix depends on **how far the mistake has travelled**.

Situation	What to do	Command
Edited a file, not committed	Discard the change	<code>git restore <file></code>
Committed locally, not pushed	Undo the commit, keep the work	<code>git reset --soft HEAD~1</code>
Pushed to a shared branch	Create a new commit that cancels it (never rewrite shared history)	<code>git revert <commit></code>
Merged a bad PR or release	Revert the merge commit	<code>git revert -m 1 <merge-commit></code>
Lost a commit or branch	Find it in the reflog and restore it	<code>git reflog</code> then <code>git branch recover <hash></code>
Bad version live in production	Redeploy the previous tag, then fix properly	<code>git checkout v1.3.2</code>
Secret (password/key) committed	Revoke the secret first , tell Security, then remove it from history	<code>git filter-repo</code> (with Security's approval)

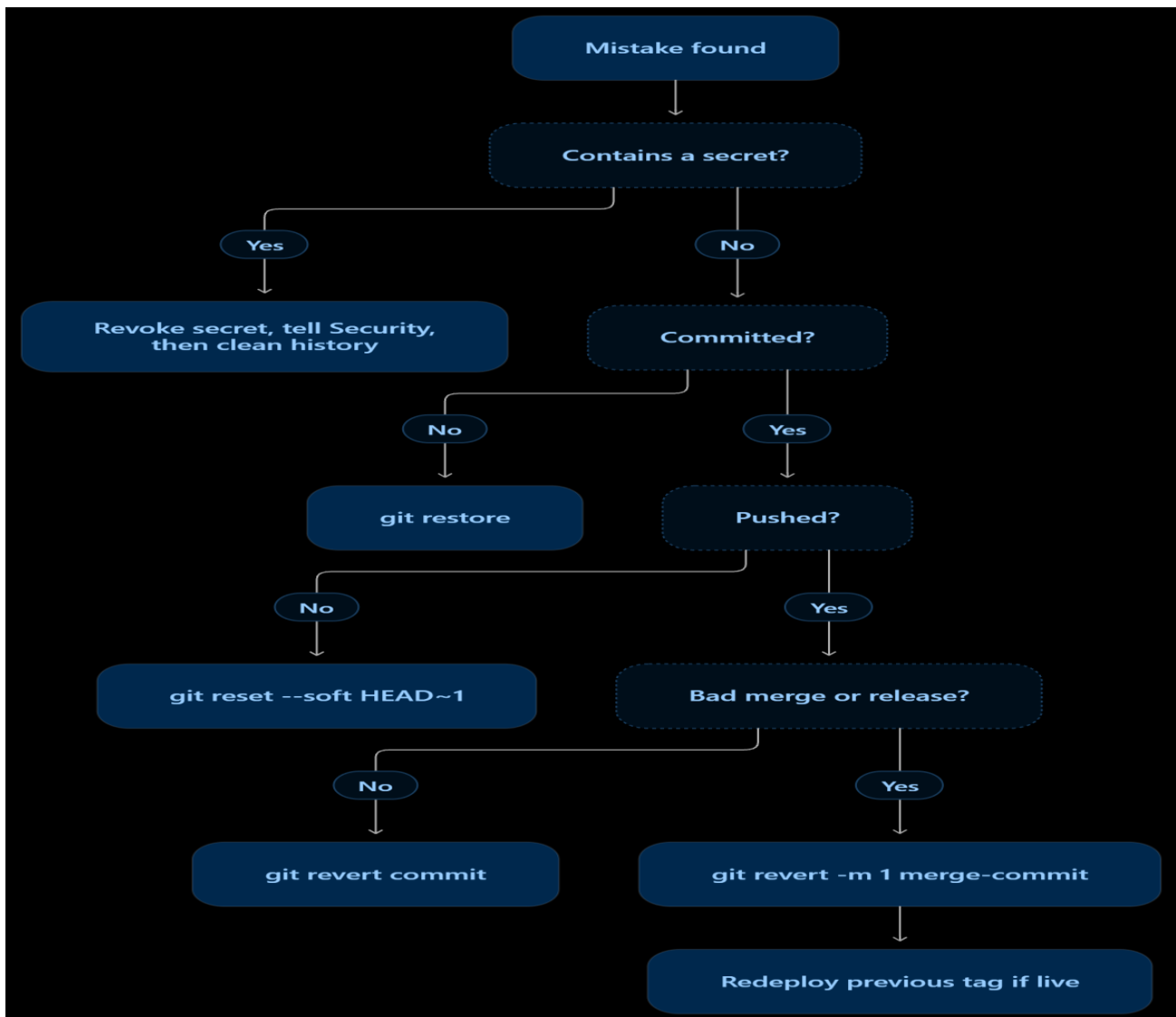


Diagram 4: Rollback decision flow

Golden rules

- **Revert, do not reset**, anything already pushed to a shared branch.
- **Never force push** to `main`, `develop` or `release/*`; branch protection blocks it.
- **A leaked secret is compromised forever**. Removing it from Git is not enough; it must be revoked or rotated.
- Every incident is logged (what, who, fix, how to prevent it).

7. Implementation Plan

The plan runs over 6 weeks (an assumption), so work is not disrupted.

Phase	Week	Activities	Output
1. Prepare	1	Agree the strategy with team leads; write the workflow guide; create the repository layout	Approved guide; repository ready
2. Set up	2	Configure branch protection, CODEOWNERS, PR template, CI checks and secret scanning	Rules enforced by the platform
3. Train	3	Short sessions on branching, commits, conflicts and recovery; cheat sheet shared	Developers trained

Phase	Week	Activities	Output
4. Pilot	4 to 5	Two modules (for example notifications and reporting) use the workflow for one release	Feedback and fixes to the guide
5. Roll out	6	All five modules switch over; old branches retired	Full adoption
6. Review	After first 2 releases	Measure results against Section 8; adjust the process	Review report

Why pilot first: testing on two lower-risk modules finds problems before they affect payments and authentication.

8. Measurable Success Criteria

Targets are illustrative (Assumption A5). Real baselines should be measured before rollout and targets adjusted.

#	Metric	How measured	Target
S1	Overwritten or lost changes	Incident reports	0 per quarter
S2	Direct pushes to main / develop	Platform logs (protected branches)	0
S3	Commits following the standard	Automated commit-message check	95% or more
S4	PRs reviewed before merging	PR records	100%
S5	Releases delivered on the planned date	Release calendar vs actual	90% or more
S6	Time to merge a PR	Platform analytics	Under 2 working days
S7	Time to recover from a bad change	Incident log	Under 1 hour
S8	Secrets found in the repository	Secret-scanning reports	0
S9	Developers trained on the workflow	Attendance records	100%

9. Recommendations and Lessons Learned

9.1 Recommendations

1. Adopt the simplified Git Flow with one repository and protected branches.
2. Enforce rules with platform settings (branch protection, CODEOWNERS, CI), not just documentation.
3. Start with a pilot on lower-risk modules before payments and authentication.
4. Keep feature branches short and PRs small.
5. Run a short recovery drill each quarter so the procedures are practised.
6. Review the process after two releases and simplify anything developers avoid.

9.2 Lessons Learned

These come from designing this solution, not from a live deployment.

- **Tools do not fix process problems.** Git alone does not stop overwriting; agreed rules and enforcement do.
- **Simplicity wins adoption.** A smaller set of branch types is easier for 50 people to follow.
- **Banking raises the bar.** Audit trails, reviews and security checks shape the design more than speed does.

- **Every model has a trade-off.** Git Flow gives structure but risks long-lived branches, which must be controlled by rule.
- **Plan recovery before you need it.** Knowing the right command for each situation prevents panic and further damage.